

Volume 06 - Runtime Ecosystem Specifications

Version v27 draft

spec-06-runtime-01-runtime-ecosystem-specification.md

Runtime Ecosystem Specification

Title: Runtime Ecosystem Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-001

Related Blueprint Documents

- MAL-V03-B03-07-001
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for the full runtime ecosystem for models, tools, plugins, connectors, providers, services, workflows, memory, and missions in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for the full runtime ecosystem for models, tools, plugins, connectors, providers, services, workflows, memory, and missions.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime MUST register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime MUST validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.

- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.
- The runtime **MUST** provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces **SHOULD** be idempotent for retry-safe commands.
- Runtime workloads **MUST** fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state **MUST** be recoverable after process restart using canonical storage records.
- Runtime overhead **SHOULD** be observable and bounded by configured limits.
- Runtime behavior **MUST** avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/runtimeecosystem/validate
- POST /runtime/runtimeecosystem/execute
- POST /runtime/runtimeecosystem/cancel
- GET /runtime/runtimeecosystem/status
- GET /runtime/runtimeecosystem/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.

- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- RuntimeEcosystemValidationRequested
- RuntimeEcosystemExecutionStarted
- RuntimeEcosystemExecutionSucceeded
- RuntimeEcosystemExecutionRejected
- RuntimeEcosystemExecutionFailed
- RuntimeEcosystemRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.
- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-001
- MAL-V03-B03-07-001
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Model Runtime Specification

Title: Model Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-002

Related Blueprint Documents

- MAL-V03-B03-10-001
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for model loading, routing, execution, fallback, evaluation hooks, and provider-independent model calls in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for model loading, routing, execution, fallback, evaluation hooks, and provider-independent model calls.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime MUST provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces SHOULD be idempotent for retry-safe commands.
- Runtime workloads MUST fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state MUST be recoverable after process restart using canonical storage records.
- Runtime overhead SHOULD be observable and bounded by configured limits.
- Runtime behavior MUST avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/modelruntime/validate
- POST /runtime/modelruntime/execute
- POST /runtime/modelruntime/cancel
- GET /runtime/modelruntime/status
- GET /runtime/modelruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- ModelRuntimeValidationRequested

- `ModelRuntimeExecutionStarted`
- `ModelRuntimeExecutionSucceeded`
- `ModelRuntimeExecutionRejected`
- `ModelRuntimeExecutionFailed`
- `ModelRuntimeRecoveryCompleted`

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-002
- MAL-V03-B03-10-001
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Tool Runtime Specification

Title: Tool Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-003

Related Blueprint Documents

- MAL-V03-B03-07-001
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for tool registration, invocation, permissions, sandboxing, result handling, and audit in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for tool registration, invocation, permissions, sandboxing, result handling, and audit.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime MUST provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces SHOULD be idempotent for retry-safe commands.
- Runtime workloads MUST fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state MUST be recoverable after process restart using canonical storage records.
- Runtime overhead SHOULD be observable and bounded by configured limits.
- Runtime behavior MUST avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/toolruntime/validate
- POST /runtime/toolruntime/execute
- POST /runtime/toolruntime/cancel
- GET /runtime/toolruntime/status
- GET /runtime/toolruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- ToolRuntimeValidationRequested

- ToolRuntimeExecutionStarted
- ToolRuntimeExecutionSucceeded
- ToolRuntimeExecutionRejected
- ToolRuntimeExecutionFailed
- ToolRuntimeRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-003
- MAL-V03-B03-07-001
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Plugin Runtime Specification

Title: Plugin Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-004

Related Blueprint Documents

- MAL-V03-B03-12-001
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for plugin loading, lifecycle, permissions, dependency checks, sandboxing, and compatibility in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for plugin loading, lifecycle, permissions, dependency checks, sandboxing, and compatibility.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime **MUST** provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces **SHOULD** be idempotent for retry-safe commands.
- Runtime workloads **MUST** fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state **MUST** be recoverable after process restart using canonical storage records.
- Runtime overhead **SHOULD** be observable and bounded by configured limits.
- Runtime behavior **MUST** avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/pluginruntime/validate
- POST /runtime/pluginruntime/execute
- POST /runtime/pluginruntime/cancel
- GET /runtime/pluginruntime/status
- GET /runtime/pluginruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- PluginRuntimeValidationRequested

- `PluginRuntimeExecutionStarted`
- `PluginRuntimeExecutionSucceeded`
- `PluginRuntimeExecutionRejected`
- `PluginRuntimeExecutionFailed`
- `PluginRuntimeRecoveryCompleted`

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-004
- MAL-V03-B03-12-001
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Connector Runtime Specification

Title: Connector Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-005

Related Blueprint Documents

- MAL-V03-B03-13-001
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for connector authentication, external system calls, sync, webhooks, retries, and rate limits in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for connector authentication, external system calls, sync, webhooks, retries, and rate limits.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime **MUST** provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces **SHOULD** be idempotent for retry-safe commands.
- Runtime workloads **MUST** fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state **MUST** be recoverable after process restart using canonical storage records.
- Runtime overhead **SHOULD** be observable and bounded by configured limits.
- Runtime behavior **MUST** avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/connectorruntime/validate
- POST /runtime/connectorruntime/execute
- POST /runtime/connectorruntime/cancel
- GET /runtime/connectorruntime/status
- GET /runtime/connectorruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- ConnectorRuntimeValidationRequested

- ConnectorRuntimeExecutionStarted
- ConnectorRuntimeExecutionSucceeded
- ConnectorRuntimeExecutionRejected
- ConnectorRuntimeExecutionFailed
- ConnectorRuntimeRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-005
- MAL-V03-B03-13-001
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Provider Runtime Specification

Title: Provider Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-006

Related Blueprint Documents

- MAL-V03-B03-11-001
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for provider routing, credentials, quotas, health, fallback, billing metadata, and compliance boundaries in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for provider routing, credentials, quotas, health, fallback, billing metadata, and compliance boundaries.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime **MUST** provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces **SHOULD** be idempotent for retry-safe commands.
- Runtime workloads **MUST** fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state **MUST** be recoverable after process restart using canonical storage records.
- Runtime overhead **SHOULD** be observable and bounded by configured limits.
- Runtime behavior **MUST** avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/providerruntime/validate
- POST /runtime/providerruntime/execute
- POST /runtime/providerruntime/cancel
- GET /runtime/providerruntime/status
- GET /runtime/providerruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- ProviderRuntimeValidationRequested

- ProviderRuntimeExecutionStarted
- ProviderRuntimeExecutionSucceeded
- ProviderRuntimeExecutionRejected
- ProviderRuntimeExecutionFailed
- ProviderRuntimeRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-006
- MAL-V03-B03-11-001
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Service Runtime Specification

Title: Service Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-007

Related Blueprint Documents

- MAL-V03-B03-07-009
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for service execution, lifecycle hooks, dependency access, isolation, and operational health in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for service execution, lifecycle hooks, dependency access, isolation, and operational health.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime **MUST** provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces **SHOULD** be idempotent for retry-safe commands.
- Runtime workloads **MUST** fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state **MUST** be recoverable after process restart using canonical storage records.
- Runtime overhead **SHOULD** be observable and bounded by configured limits.
- Runtime behavior **MUST** avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/serviceruntime/validate
- POST /runtime/serviceruntime/execute
- POST /runtime/serviceruntime/cancel
- GET /runtime/serviceruntime/status
- GET /runtime/serviceruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- ServiceRuntimeValidationRequested

- ServiceRuntimeExecutionStarted
- ServiceRuntimeExecutionSucceeded
- ServiceRuntimeExecutionRejected
- ServiceRuntimeExecutionFailed
- ServiceRuntimeRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-007
- MAL-V03-B03-07-009
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Workflow Runtime Specification

Title: Workflow Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-008

Related Blueprint Documents

- MAL-V03-B03-08-002
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for workflow execution, state transitions, approval gates, recovery, and scheduling integration in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for workflow execution, state transitions, approval gates, recovery, and scheduling integration.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime **MUST** provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces **SHOULD** be idempotent for retry-safe commands.
- Runtime workloads **MUST** fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state **MUST** be recoverable after process restart using canonical storage records.
- Runtime overhead **SHOULD** be observable and bounded by configured limits.
- Runtime behavior **MUST** avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/workflowruntime/validate
- POST /runtime/workflowruntime/execute
- POST /runtime/workflowruntime/cancel
- GET /runtime/workflowruntime/status
- GET /runtime/workflowruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- WorkflowRuntimeValidationRequested

- WorkflowRuntimeExecutionStarted
- WorkflowRuntimeExecutionSucceeded
- WorkflowRuntimeExecutionRejected
- WorkflowRuntimeExecutionFailed
- WorkflowRuntimeRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-008
- MAL-V03-B03-08-002
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Memory Runtime Specification

Title: Memory Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-009

Related Blueprint Documents

- MAL-V03-B03-04-005
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for memory read/write runtime behavior, scopes, provenance, retention, recall, and correction in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for memory read/write runtime behavior, scopes, provenance, retention, recall, and correction.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime MUST provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces SHOULD be idempotent for retry-safe commands.
- Runtime workloads MUST fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state MUST be recoverable after process restart using canonical storage records.
- Runtime overhead SHOULD be observable and bounded by configured limits.
- Runtime behavior MUST avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/memoryruntime/validate
- POST /runtime/memoryruntime/execute
- POST /runtime/memoryruntime/cancel
- GET /runtime/memoryruntime/status
- GET /runtime/memoryruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- MemoryRuntimeValidationRequested

- MemoryRuntimeExecutionStarted
- MemoryRuntimeExecutionSucceeded
- MemoryRuntimeExecutionRejected
- MemoryRuntimeExecutionFailed
- MemoryRuntimeRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-009
- MAL-V03-B03-04-005
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Mission Runtime Specification

Title: Mission Runtime Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-010

Related Blueprint Documents

- MAL-V03-B03-07-008
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for mission execution, mission graph state, ownership, progress, review, and completion evidence in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for mission execution, mission graph state, ownership, progress, review, and completion evidence.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime MUST provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces SHOULD be idempotent for retry-safe commands.
- Runtime workloads MUST fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state MUST be recoverable after process restart using canonical storage records.
- Runtime overhead SHOULD be observable and bounded by configured limits.
- Runtime behavior MUST avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/missionruntime/validate
- POST /runtime/missionruntime/execute
- POST /runtime/missionruntime/cancel
- GET /runtime/missionruntime/status
- GET /runtime/missionruntime/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- MissionRuntimeValidationRequested

- MissionRuntimeExecutionStarted
- MissionRuntimeExecutionSucceeded
- MissionRuntimeExecutionRejected
- MissionRuntimeExecutionFailed
- MissionRuntimeRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-010
- MAL-V03-B03-07-008
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Runtime Lifecycle Specification

Title: Runtime Lifecycle Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-011

Related Blueprint Documents

- MAL-V03-B03-07-002
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for runtime lifecycle states, boot, readiness, degradation, maintenance, shutdown, and compatibility gates in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime lifecycle states, boot, readiness, degradation, maintenance, shutdown, and compatibility gates.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime **MUST** provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces **SHOULD** be idempotent for retry-safe commands.
- Runtime workloads **MUST** fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state **MUST** be recoverable after process restart using canonical storage records.
- Runtime overhead **SHOULD** be observable and bounded by configured limits.
- Runtime behavior **MUST** avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/runtimelifecycle/validate
- POST /runtime/runtimelifecycle/execute
- POST /runtime/runtimelifecycle/cancel
- GET /runtime/runtimelifecycle/status
- GET /runtime/runtimelifecycle/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- RuntimeLifecycleValidationRequested

- RuntimeLifecycleExecutionStarted
- RuntimeLifecycleExecutionSucceeded
- RuntimeLifecycleExecutionRejected
- RuntimeLifecycleExecutionFailed
- RuntimeLifecycleRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-011
- MAL-V03-B03-07-002
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Runtime Permissions Specification

Title: Runtime Permissions Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-012

Related Blueprint Documents

- MAL-V03-B03-07-012
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for runtime permissions for tools, plugins, connectors, providers, memory, workflows, and protected actions in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime permissions for tools, plugins, connectors, providers, memory, workflows, and protected actions.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime MUST provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces SHOULD be idempotent for retry-safe commands.
- Runtime workloads MUST fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state MUST be recoverable after process restart using canonical storage records.
- Runtime overhead SHOULD be observable and bounded by configured limits.
- Runtime behavior MUST avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/runtimepermissions/validate
- POST /runtime/runtimepermissions/execute
- POST /runtime/runtimepermissions/cancel
- GET /runtime/runtimepermissions/status
- GET /runtime/runtimepermissions/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- RuntimePermissionsValidationRequested

- RuntimePermissionsExecutionStarted
- RuntimePermissionsExecutionSucceeded
- RuntimePermissionsExecutionRejected
- RuntimePermissionsExecutionFailed
- RuntimePermissionsRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-012
- MAL-V03-B03-07-012
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Runtime Observability Specification

Title: Runtime Observability Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-013

Related Blueprint Documents

- MAL-V03-B03-07-013
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for runtime metrics, logs, traces, alerts, dashboards, health, and operator diagnostics in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime metrics, logs, traces, alerts, dashboards, health, and operator diagnostics.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime **MUST** provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces **SHOULD** be idempotent for retry-safe commands.
- Runtime workloads **MUST** fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state **MUST** be recoverable after process restart using canonical storage records.
- Runtime overhead **SHOULD** be observable and bounded by configured limits.
- Runtime behavior **MUST** avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/runtimeobservability/validate
- POST /runtime/runtimeobservability/execute
- POST /runtime/runtimeobservability/cancel
- GET /runtime/runtimeobservability/status
- GET /runtime/runtimeobservability/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- RuntimeObservabilityValidationRequested

- RuntimeObservabilityExecutionStarted
- RuntimeObservabilityExecutionSucceeded
- RuntimeObservabilityExecutionRejected
- RuntimeObservabilityExecutionFailed
- RuntimeObservabilityRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-013
- MAL-V03-B03-07-013
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Runtime Failure Recovery Specification

Title: Runtime Failure Recovery Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-014

Related Blueprint Documents

- MAL-V03-B03-07-014
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for runtime failure detection, retry, rollback, circuit breaking, quarantine, and recovery evidence in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime failure detection, retry, rollback, circuit breaking, quarantine, and recovery evidence.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime MUST provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces SHOULD be idempotent for retry-safe commands.
- Runtime workloads MUST fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state MUST be recoverable after process restart using canonical storage records.
- Runtime overhead SHOULD be observable and bounded by configured limits.
- Runtime behavior MUST avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/runtimefailure/recovery/validate
- POST /runtime/runtimefailure/recovery/execute
- POST /runtime/runtimefailure/recovery/cancel
- GET /runtime/runtimefailure/recovery/status
- GET /runtime/runtimefailure/recovery/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- RuntimeFailureRecoveryValidationRequested

- RuntimeFailureRecoveryExecutionStarted
- RuntimeFailureRecoveryExecutionSucceeded
- RuntimeFailureRecoveryExecutionRejected
- RuntimeFailureRecoveryExecutionFailed
- RuntimeFailureRecoveryRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-014
- MAL-V03-B03-07-014
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.

Runtime Testing Specification

Title: Runtime Testing Specification

Version: v27 draft

Status: Draft

Specification ID: MAL-V06-RUNTIME-SPEC-015

Related Blueprint Documents

- MAL-V03-B03-07-019
- MAL-V03-B03-07-README
- MAL-V03-B03-07-001
- MAL-V03-B03-01-004
- MAL-V05-STD-008
- MAL-V05-STD-012

Purpose

Define executable requirements for runtime unit, integration, security, performance, recovery, and acceptance testing in Mariam AI Enterprise OS.

Scope

This specification covers functional behavior, non-functional expectations, inputs, outputs, interfaces, events, storage, security, observability, failure handling, recovery, tests, and implementation notes for runtime unit, integration, security, performance, recovery, and acceptance testing.

Responsibilities

- Convert runtime ecosystem blueprints into implementation-ready requirements.
- Govern invocation, execution, permissions, state, recovery, and evidence for runtime workloads.
- Preserve compatibility with Enterprise Core runtime manager, identity access, event bus, storage, security, and observability.
- Prevent unreviewed tools, plugins, connectors, providers, workflows, missions, or memory actions from becoming authoritative.

Functional Requirements

- The runtime **MUST** register executable units with owner, version, type, permission scope, lifecycle state, and compatibility metadata.
- The runtime **MUST** validate caller identity, tenant context, permission policy, configuration, and dependency health before execution.
- The runtime **MUST** isolate tool, plugin, connector, provider, model, service, workflow, memory, and mission executions according to risk.
- The runtime **MUST** emit canonical events for scheduling, start, success, rejection, failure, timeout, retry, rollback, and recovery.
- The runtime **MUST** support dry-run or validation-only execution where protected side effects are possible.

- The runtime MUST provide deterministic status for active, queued, degraded, failed, quarantined, and stopped runtime objects.

Non-Functional Requirements

- Runtime interfaces SHOULD be idempotent for retry-safe commands.
- Runtime workloads MUST fail closed when permissions, configuration, provider credentials, or policy status are missing.
- Runtime state MUST be recoverable after process restart using canonical storage records.
- Runtime overhead SHOULD be observable and bounded by configured limits.
- Runtime behavior MUST avoid leaking secrets, external payloads, prompts, credentials, or classified data in logs or events.

Inputs

- Runtime command, workload definition, tool call, plugin request, connector request, model invocation, provider route, workflow state, memory command, or mission command.
- Caller identity, tenant context, role, permission grant, policy state, and approval evidence.
- Configuration values, runtime registry metadata, dependency health, quota status, and resource availability.
- Input payloads, schemas, files, messages, embeddings, memory references, or external provider references.

Outputs

- Runtime execution result, rejection result, retry state, failure report, rollback status, or recovery status.
- Runtime events, audit records, metrics, traces, logs, and health signals.
- Updated runtime state, invocation history, quota records, dependency records, or execution evidence.
- Sanitized diagnostics for operators and downstream reviewers.

Public Interfaces

- POST /runtime/runtimetesting/validate
- POST /runtime/runtimetesting/execute
- POST /runtime/runtimetesting/cancel
- GET /runtime/runtimetesting/status
- GET /runtime/runtimetesting/events

Internal Interfaces

- Enterprise Core runtime manager and runtime registry.
- Identity access authorization guard.
- Event bus publisher and subscriber APIs.
- Configuration and secret providers.
- Core storage, audit store, and observability pipeline.
- Resource manager, health manager, workflow engine, provider registry, and capability registry where applicable.

Events

- RuntimeTestingValidationRequested

- RuntimeTestingExecutionStarted
- RuntimeTestingExecutionSucceeded
- RuntimeTestingExecutionRejected
- RuntimeTestingExecutionFailed
- RuntimeTestingRecoveryCompleted

Storage Requirements

- Persist runtime definitions, execution records, state transitions, permission decisions, quota usage, and audit references.
- Store external provider references and connector sync cursors without storing secrets in plaintext.
- Store derived runtime views separately so they can be rebuilt from canonical records.
- Maintain retention metadata for execution evidence, logs, traces, and failure reports.

Security Requirements

- Enforce least privilege for every runtime invocation and dependency access.
- Deny execution when runtime permissions, tenant isolation, data classification, or approval requirements are not satisfied.
- Redact secrets, credentials, prompts, files, tokens, and external payload fragments from observability output.
- Audit privileged runtime execution, denied execution, provider credential access, connector access, rollback, and exception usage.
- Sandbox untrusted plugins, connectors, tools, or external payload processors.

Observability Requirements

- Emit metrics for validation latency, queue time, execution time, success rate, rejection rate, error rate, retry count, timeout count, and recovery count.
- Attach correlation IDs to events, logs, traces, audit records, and storage records.
- Provide health and readiness indicators for runtime dependencies.
- Provide operator diagnostics with redacted payloads and actionable failure categories.

Failure Modes

- Missing permission, missing tenant context, denied policy, expired approval, or invalid runtime definition.
- Provider outage, connector timeout, plugin crash, model failure, tool failure, workflow deadlock, memory write failure, or storage write failure.
- Quota exceeded, rate limit exceeded, dependency unavailable, configuration mismatch, or schema validation failure.
- Partial side effect occurred before failure.

Recovery Rules

- Retry idempotent executions with bounded attempts and backoff.
- Roll back or compensate side effects when a runtime operation fails after partial progress.
- Quarantine unsafe runtime definitions, connectors, plugins, tools, or provider routes until review.
- Rebuild derived runtime state from canonical execution and registry records.

- Escalate repeated high-impact runtime failures to governance review.

Test Requirements

- Unit tests for validation, permission checks, lifecycle transitions, event generation, and failure mapping.
- Integration tests across runtime registry, identity access, event bus, storage, configuration, observability, and providers/connectors/plugins where applicable.
- Security tests for sandboxing, tenant isolation, secret redaction, denied execution, and protected actions.
- Recovery tests for retry, rollback, timeout, restart, provider outage, connector outage, and event publication failure.
- Acceptance tests proving required events, metrics, storage records, and audit evidence exist.

Acceptance Criteria

- The specification is executable enough to guide implementation without hidden runtime assumptions.
- Public and internal interfaces, event behavior, storage behavior, security behavior, and recovery behavior are explicit.
- Runtime actions remain traceable to caller intent, permission, configuration, policy, and execution evidence.
- Tests can verify success, denial, failure, retry, rollback, and recovery behavior.

Implementation Notes

- Start with validation and read-only runtime status before enabling side-effecting execution.
- Keep runtime definitions separate from runtime instances and execution records.
- Use structured schemas for runtime commands, results, events, and failure records.
- Treat every external provider, plugin, connector, and tool as untrusted until policy and permissions prove otherwise.

Traceability

- MAL-V06-RUNTIME-SPEC-015
- MAL-V03-B03-07-019
- MAL-V03-B03-07-README
- MAL-V05-STD-008
- MAL-V05-STD-012
- MAL-V05-STD-013
- MAL-V06-CORE-SPEC-003
- MAL-V06-CORE-SPEC-008

Version History

Version	Date	Status	Notes
v27 draft	2026-06-29	Draft	Initial Runtime Ecosystem specification.